OmniLink Tetris AI Agent System

Complete Technical Documentation and Deployment Guide

Project Type: AI-Augmented Game Environment with Browser-Based Agent Integration
Target Platform: OmniLink Browser Tool Environment
Date: March 10, 2026
Author: Technical Documentation Team

Executive Summary

The OmniLink Tetris project is a sophisticated AI gaming system that combines a traditional Pygame-based Tetris game engine with an advanced artificial intelligence agent running in a web browser. The system uses HTTP REST APIs and MQTT messaging to create a real-time communication bridge between the Python game environment and TypeScript-based AI agents that run within the OmniLink platform's browser environment.

This project demonstrates production-grade software architecture patterns including service-oriented design, event-driven systems, and intelligent agent behavior using advanced heuristic algorithms. The AI agent is capable of playing Tetris at superhuman speeds by leveraging the Pierre Dellacherie algorithm with macro-action queuing.

Table of Contents

- Introduction and Architecture Overview
- Core Components and Technologies
- System Architecture and Communication Flow
- AI Strategy and Algorithm Details
- Installation and Setup Requirements
- Running the System Step-by-Step
- Configuration and Customization
- Troubleshooting and Common Issues
- Technical Details and Implementation Notes

---

1. Introduction and Architecture Overview

1.1 What is This Project?

The OmniLink Tetris project is an **intelligent game-playing system** consisting of three interconnected components:

1. **Game Engine (tetris.py):** A fully-featured Tetris implementation built with Pygame, including standard game mechanics, scoring, lives system, and time-based difficulty scaling
2. **Backend Bridge (server_wrapper.py):** A Python HTTP/MQTT server that exposes game state via REST APIs and accepts control commands, acting as middleware between the game and AI agents
3. **AI Agents (TypeScript files):** Browser-based intelligent agents that analyze game state, compute optimal moves using advanced algorithms, and send macro-action sequences to control gameplay

1.2 Why This Architecture?

This separation of concerns enables several powerful capabilities:

- **Platform Independence:** AI agents run in any modern browser, including the OmniLink platform's isolated tool environment
- **Real-Time Communication:** HTTP polling and MQTT pub/sub patterns enable sub-100ms response times

- **Agent Flexibility:** Multiple agent strategies can be developed and tested without modifying game code
- **Scalability:** The REST API architecture allows multiple agents or monitoring tools to connect simultaneously
- **Production Deployment:** The system demonstrates enterprise-grade patterns suitable for cloud deployment

1.3 Key Features

Game Mechanics:

- Standard Tetris rotation, movement, and line-clearing rules
- 15-life system with board wipe on game-over (not instant death)
- Exponential difficulty scaling based on playtime (gravity accelerates over time)
- High score persistence across sessions
- Real-time scoring and level progression

AI Capabilities:

- Macro-action queuing system for superhuman gameplay speeds
- Pierre Dellacherie heuristic evaluation with custom weight tuning
- Full rotation and position space exploration for each piece
- Adaptive decision-making with stuck-state detection
- Real-time telemetry and performance logging

Communication Infrastructure:

- RESTful HTTP API on port 5001 for state queries and action submission
- MQTT messaging on WebSocket port 9001 for pause/resume control
- JSON-based data serialization for cross-language compatibility
- Version tracking to prevent stale data processing

---

2. Core Components and Technologies

2.1 Technology Stack

Component	Technology	Purpose
Game Engine	Python 3.x + Pygame	Graphics, physics, game logic
Backend Server	Python HTTP + MQTT	API endpoints, message routing
AI Agents	TypeScript (ESM)	Decision algorithms, browser execution
Communication	REST + MQTT	Real-time data exchange
Data Format	JSON	State serialization

Table 1: Technology stack breakdown

2.2 File Structure

Python Backend:

- `tetris.py` (15,796 characters): Core game engine with complete Tetris implementation
- `server_wrapper.py` (8,808 characters): HTTP REST API server and MQTT bridge

TypeScript Agents:

- `advanced_agent.ts` (14,564 characters): Production-grade AI using Pierre Dellacherie algorithm with macro-action queuing
- `agent.ts` (14,817 characters): Basic agent implementation for standard gameplay (struggles with high-speed gravity)

Documentation:

- `README.md` (3,614 characters): Quick-start guide and architecture overview

2.3 Python Dependencies

- **pygame:** Graphics rendering, event handling, game loop management
- **paho-mqtt:** MQTT client library for publish/subscribe messaging
- **http.server:** Built-in Python HTTP server for REST API endpoints
- **threading:** Concurrent execution of HTTP server, MQTT loop, and game loop
- **json:** Data serialization for API responses

2.4 TypeScript/Browser Requirements

- **Node.js runtime:** Required for `npx ts-node` execution during development
- **TypeScript compiler:** Transpiles `.ts` files to JavaScript
- **Fetch API:** Modern browser HTTP client for polling game state
- **MQTT.js library:** WebSocket-based MQTT client for browser environments
- **OmniLink platform:** Isolated browser tool environment where agents execute

---

3. System Architecture and Communication Flow

3.1 High-Level Architecture Diagram

```
┌──────────────
```

```
│  │ HTTP GET/POST │ MQTT WS │
└──────────────────────────┴──────────────────────────────────┴──────
──────────────────────────┘
│  │
┌──────────────▼──────────────┐  ┌──────────────▼──────────────┐
│ HTTP REST API │  │ MQTT Broker │
│ (Port 5001) │  │ (Port 9001 WS) │
│ - GET /data │  │ - olinkcommands │
│ - POST /callback │  │ - olinkcontext │
└──────────────┬──────────────┘  └──────────────┬──────────────┘
│  │
┌──────────────▼──────────────────────────────▼──────────────
──┐
│ server_wrapper.py (Backend Bridge) │
│ - Manages game state synchronization │
│ - Routes commands to game engine │
│ - Publishes telemetry every 20s │
└──────────────┬──────────────────────────────────────────────
┘
│
┌──────────────▼──────────────────────────────────────────────
──┐
│ tetris.py (Pygame Game Engine) │
│ - Renders graphics at 60 FPS │
│ - Processes gravity and collisions │
│ - Consumes pendingActions queue │
└──────────────────────────────────────────────────────────────
┘
```

3.2 Communication Protocols

HTTP REST API (Port 5001):

Endpoint	Method	Purpose
/data	GET	Retrieve current game state JSON
/callback	POST	Submit action array to game
/start	POST	Trigger game start from TITLE screen

Table 2: HTTP REST API endpoints

MQTT Topics:

Topic	Direction	Purpose
olinkcommands	Agent → Game	Pause/resume control commands
olinkcontext	Game → Agent	Telemetry data every 20s

Table 3: MQTT messaging topics

3.3 Data Flow Sequence

Typical Gameplay Loop (60ms cycle):

1. **Agent polls** HTTP GET to `/data` endpoint
2. **Server responds** with JSON containing board state, current piece, next piece, score, lives, time
3. **Agent evaluates** all possible rotations and positions for current piece using heuristic function
4. **Agent computes** macro-action sequence (e.g., `["ROTATE", "ROTATE", "RIGHT", "RIGHT", "DROP"]`)
5. **Agent submits** HTTP POST to `/callback` with action array
6. **Server queues** actions into `game.pendingActions` list
7. **Game engine** drains one action every 40ms from queue (25 actions/second)
8. **Piece executes** entire movement sequence before gravity can interrupt
9. **Cycle repeats** when new piece spawns

---

4. AI Strategy and Algorithm Details

4.1 Pierre Dellacherie Algorithm Overview

The advanced agent implements the **Pierre Dellacherie algorithm**, recognized as one of the most effective one-piece Tetris AI strategies[cite:11]. This heuristic-based approach evaluates board states using multiple weighted features.

Core Algorithm Steps:

1. For each possible rotation of the current piece (I-piece has 2, T-piece has 4, etc.)
2. For each valid horizontal position across the board
3. Simulate hard-drop placement on a cloned board
4. Calculate line clears and resulting board configuration
5. Score the outcome using weighted heuristic function
6. Select the rotation and position with the highest score
7. Generate macro-action sequence to achieve that placement
8. Submit entire sequence to game engine

4.2 Heuristic Evaluation Function

The agent evaluates board states using four weighted features:

Feature	Weight	Description
Lines Cleared	+500	Reward for eliminating rows
Holes Created	-400	Penalty for empty cells below filled cells
Aggregate Height	-30	Penalty for tall column heights
Bumpiness	-50	Penalty for uneven column heights

Table 4: Heuristic evaluation weights

Evaluation Formula:

$$\text{Score} = 500 \times L - 400 \times H - 30 \times A - 50 \times B$$

Where:

- L = number of lines cleared
- H = total holes (empty cells with filled cells above)
- A = sum of all column heights
- B = sum of absolute height differences between adjacent columns

4.3 Macro-Action System

Problem: At high speeds (t=300s, gravity drops every 0.05s), standard HTTP polling (60ms cycle) cannot respond fast enough. By the time the agent decides on a move and sends one action, gravity has already moved the piece multiple rows down.

Solution: The macro-action architecture queues an entire movement sequence that executes at 40ms/action (25 actions/second), faster than gravity can interrupt.

Example Macro-Action Sequence:

```
// Agent decides T-piece should rotate twice and move right 3 columns
const actions = ["ROTATE", "ROTATE", "RIGHT", "RIGHT", "RIGHT",
"DROP"];
// Entire sequence sent in one HTTP POST
// Game drains queue at 25 actions/second
// Piece teleports to destination before gravity interferes
```

4.4 Stuck State Detection

The agent includes adaptive logic to handle unexpected board states:

- Tracks piece position every frame
- If piece hasn't moved for 20 consecutive frames, triggers recomputation
- Recalculates best placement using current board state
- Generates fresh macro-action sequence
- Prevents agent from repeating failed strategies

---

5. Installation and Setup Requirements

5.1 System Prerequisites

Operating System:

- Linux (Ubuntu 20.04+, Debian 11+)
- macOS (10.15+)
- Windows 10/11 with WSL2 recommended

Required Software:

- Python 3.8 or higher
- pip package manager
- Node.js 16.x or higher
- npm or npx (comes with Node.js)
- MQTT broker (Mosquitto recommended)

5.2 Python Environment Setup

Step 1: Install Python dependencies

```
pip install pygame paho-mqtt
```

Step 2: Verify Pygame installation

```
python -c "import pygame; print(pygame.ver)"
# Expected output: 2.x.x
```

Step 3: Verify paho-mqtt installation

```
python -c "import paho.mqtt.client as mqtt; print('MQTT OK')"
# Expected output: MQTT OK
```

5.3 MQTT Broker Setup

Option A: Install Mosquitto (Recommended)

Ubuntu/Debian:
```
sudo apt-get update
sudo apt-get install mosquitto mosquitto-clients
```

macOS (Homebrew):
```
brew install mosquitto
brew services start mosquitto
```

Windows:

- Download installer from https://mosquitto.org/download/

- Install with default settings
- Add WebSocket listener configuration

Configure WebSocket Support:

Edit `/etc/mosquitto/mosquitto.conf` (Linux/Mac) or `C:\Program Files\mosquitto\mosquitto.conf` (Windows):

```
listener 1883
protocol mqtt

listener 9001
protocol websockets
```

```
Restart Mosquitto:
sudo systemctl restart mosquitto # Linux
brew services restart mosquitto # macOS
```

Option B: Use Public MQTT Broker

For testing purposes, you can use a public broker like `test.mosquitto.org`, but modify connection settings in `server_wrapper.py`.

5.4 TypeScript Environment Setup

Step 1: Install Node.js

Download from https://nodejs.org/ or use package manager:

Ubuntu/Debian

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
```

macOS

```
brew install node
```

Step 2: Verify installation

```
node --version # Should show v16.x or higher
npm --version # Should show 8.x or higher
```

Step 3: Install TypeScript globally (optional)

```
npm install -g typescript ts-node
```

5.5 OmniLink Platform Considerations

For Browser Execution:

When running TypeScript agents in the OmniLink browser environment, the `.ts` files are executed as **ESM modules** in an isolated worker context. Key differences from local development:

- No file system access
- No Node.js-specific APIs
- MQTT.js must be loaded via global variable or module import
- Console logging appears in browser DevTools
- HTTP requests must target accessible endpoints (localhost works if game runs locally)

Global Functions:

The agents expose global functions for external control:

```
globalThis.pauseGame(); // Sends MQTT pause command
globalThis.resumeGame(); // Sends MQTT resume command
```

These functions can be called from the OmniLink platform's console or other tools.

---

6. Running the System Step-by-Step

6.1 Complete Startup Procedure

Terminal 1: Start MQTT Broker (if not running as service)

```
mosquitto -c /etc/mosquitto/mosquitto.conf -v
```

Expected output:
```
1678483200: mosquitto version 2.x starting
```

1678483200: Opening ipv4 listen socket on port 1883.
1678483200: Opening websockets listen socket on port 9001.

Terminal 2: Start Game Backend

cd /path/to/project
python server_wrapper.py

Expected output:
[HTTP] API on port 5001
[MQTT] Connected to localhost:1883
[MQTT] Subscribed to olinkcommands
[Game] Initialising Tetris...
[Game] Ready. waiting for agent commands on port 5001

You should see a Pygame window open with Tetris board in TITLE state (waiting to start).

Terminal 3: Start Advanced AI Agent

cd /path/to/project
npx ts-node advanced_agent.ts

Expected output:

Advanced Tetris Agent (Macro arrays)

[CONFIG] API: http://localhost:5001 (poll every 60ms)
[CONFIG] MQTT: ws://localhost:9001 (topic=olinkcommands)
[WEIGHTS] lines=500, holes=-400, height=-30, bumpiness=-50
[INFO] globalThis.pauseGame() / resumeGame() available

[GAME] State: PLAY
[AI] New piece=T best rot=2 x=3 score=1240
[AI] Macro Actions: ROTATE → ROTATE → RIGHT → DROP
[GAME] Score: 0→100 (+100)
[AI] New piece=I best rot=0 x=9 score=2180
[AI] Macro Actions: RIGHT → RIGHT → RIGHT → RIGHT → RIGHT →

DROP
[GAME] Lines cleared: 1 (total)
[GAME] Score: 100→400 (+300)

The agent will immediately connect to the game and start playing autonomously.

6.2 Verification Steps

Check 1: HTTP API Accessibility

Open browser or use curl:
curl http://localhost:5001/data

Expected: JSON response with game state (command, payload, version)

Check 2: MQTT Connection

Subscribe to telemetry topic:
mosquitto_sub -h localhost -p 1883 -t olinkcontext -v

Expected: JSON messages every 20 seconds with game summary

Check 3: Agent Performance

Watch the Pygame window - you should see:

- Pieces spawning at top
- Pieces immediately moving to optimal positions
- Lines clearing rapidly
- Score increasing
- No manual intervention required

6.3 OmniLink Platform Execution

To run agents in OmniLink browser environment:

1. Ensure game backend is running on accessible network address
2. Update `APIURL` in TypeScript file if not localhost
3. Load .ts file into OmniLink tool environment
4. Agent begins execution automatically in isolated worker
5. Monitor via browser console (F12 DevTools)

Network Configuration:

If OmniLink runs on different machine than game:

```
// In advanced_agent.ts, update:
const APIURL = 'http://192.168.1.100:5001'; // Game server IP
const MQTTWSURL = 'ws://192.168.1.100:9001'; // MQTT broker IP
```

CORS Considerations:

The Python server includes CORS headers to allow browser access:

```
self.send_header('Access-Control-Allow-Origin', '*')
self.send_header('Access-Control-Allow-Methods', 'GET, POST, OPTIONS')
```

---

7. Configuration and Customization

7.1 Game Difficulty Tuning

Modify tetris.py:

```
# Initial lives (default: 15)
self.lives = 15 # Change to increase/decrease difficulty

# Gravity formula (exponential difficulty scaling)
def gravity(self):
return max(0.05, 1.0 * (0.92 ** (self.playtime / 15.0)))
# 0.05 = minimum interval (0.05s = 20 drops/second at max speed)
# 1.0 = initial interval (1s drop at start)
# 0.92 = decay rate (lower = faster difficulty increase)
# 15.0 = time scaling factor (seconds per difficulty step)
```

Example Modifications:

- Easier game: `self.lives = 30`, decay rate `0.95`

- Harder game: `self.lives = 5`, decay rate `0.88`
- Constant speed: `return 0.5` (no exponential scaling)

7.2 Agent Heuristic Weights

Modify advanced_agent.ts:

```
// Current weights (tuned for balanced play)
const W_LINES = 500; // Reward for clearing lines
const W_HOLES = -400; // Penalty for creating holes
const W_HEIGHT = -30; // Penalty for tall columns
const W_BUMPINESS = -50; // Penalty for uneven surface

// Aggressive clearing strategy (prioritize lines)
const W_LINES = 800;
const W_HOLES = -200;

// Defensive strategy (avoid holes, keep flat)
const W_HOLES = -600;
const W_BUMPINESS = -100;
```

Weight Optimization:

The weights can be tuned using particle swarm optimization (PSO) or genetic algorithms. The current values are manually calibrated for strong performance.

7.3 Polling and Action Speed

Agent Polling Rate (advanced_agent.ts):

```
const POLL_DELAY_MS = 60; // Poll every 60ms (16.7 Hz)

// Faster polling (may increase HTTP load)
const POLL_DELAY_MS = 40; // 25 Hz

// Slower polling (reduced CPU usage)
const POLL_DELAY_MS = 100; // 10 Hz
```

Action Drain Rate (tetris.py):

Current: drain one action every 40ms

if self.pendingActions and self.aiActionTimer >= 0.04:
self.aiActionTimer = 0.0
action = self.pendingActions.pop(0)

Faster execution (50 actions/second)

if self.pendingActions and self.aiActionTimer >= 0.02:

Slower execution (10 actions/second)

if self.pendingActions and self.aiActionTimer >= 0.10:

7.4 MQTT Telemetry Frequency

Modify server_wrapper.py:

PUBLISH_EVERY = 20 # Publish telemetry every 20 seconds

More frequent updates

PUBLISH_EVERY = 5 # Every 5 seconds

Less frequent updates

PUBLISH_EVERY = 60 # Every minute

7.5 Logging Verbosity

Agent Logging (advanced_agent.ts):

```
const LOG_DECISION = true; // Log best placement decisions
const LOG_ACTION = true; // Log macro-action sequences
const LOG_EVENTS = true; // Log score/level/lives changes
const LOG_IDLE = false; // Log idle polling (very verbose)
const LOG_MQTT = true; // Log MQTT connection events
const LOG_ERRORS = true; // Log error messages

// Quiet mode (only errors)
const LOG_DECISION = false;
const LOG_ACTION = false;
const LOG_EVENTS = false;
```

---

8. Troubleshooting and Common Issues

8.1 Game Backend Issues

Problem: "Address already in use" error on port 5001

Solution:

```
# Find process using port 5001
lsof -i :5001 # Linux/Mac
netstat -ano | find "5001" # Windows

# Kill existing process
kill -9 <PID> # Linux/Mac
taskkill /PID <PID> /F # Windows
```

Or change port in server_wrapper.py

HTTP_PORT = 5002 # Use different port

Problem: Pygame window not appearing

Solution:

Check display environment variable (Linux)

echo $DISPLAY # Should show :0 or :1

If empty, set it

export DISPLAY=:0

Check Pygame installation

python -c "import pygame; pygame.init()"

Problem: "ModuleNotFoundError: No module named 'paho'"

Solution:
pip install paho-mqtt

Or with specific version

pip install paho-mqtt==1.6.1

8.2 MQTT Connection Issues

Problem: Agent cannot connect to MQTT broker

Solution:

1. Verify Mosquitto is running:
   sudo systemctl status mosquitto
2. Check WebSocket listener configuration:
   grep -A 2 "listener 9001" /etc/mosquitto/mosquitto.conf
   # Should show: protocol websockets
3. Test WebSocket connectivity:
   curl -i -N -H "Connection: Upgrade"
   -H "Upgrade: websocket"
   http://localhost:9001/mqtt
   # Should return 101 Switching Protocols
4. Check firewall rules:
   sudo ufw allow 9001/tcp # Ubuntu
   sudo firewall-cmd --add-port=9001/tcp # CentOS

Problem: MQTT messages not received by game

Solution:

- Verify topic names match exactly (`olinkcommands`)
- Check MQTT broker logs: `tail -f /var/log/mosquitto/mosquitto.log`
- Test with mosquitto_pub:
  mosquitto_pub -h localhost -p 1883 -t olinkcommands -m
  '{"command":"pause"}'

8.3 Agent Execution Issues

Problem: "npx: command not found"

Solution:

Install Node.js (includes npx)

Ubuntu/Debian

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs

Verify installation

npx --version

Problem: TypeScript compilation errors

Solution:

Install TypeScript dependencies

npm install -g typescript ts-node @types/node

Or run with explicit TypeScript path

./node_modules/.bin/ts-node advanced_agent.ts

Problem: Agent receives "HTTP 503: Game not ready"

Solution:

- Ensure `server_wrapper.py` is running
- Wait 2-3 seconds after starting backend for game initialization
- Check game window is visible (not minimized)

8.4 Performance Issues

Problem: Agent too slow, pieces fall before decisions

Solution:

- Reduce polling delay: `POLL_DELAY_MS = 40`
- Simplify evaluation (disable bumpiness calculation)

- Use basic agent instead of advanced agent initially
- Check CPU usage - may need faster hardware

Problem: Game stutters or lags

Solution:

In tetris.py, reduce FPS if system is slow

FPS = 30 # Instead of 60

Disable anti-aliasing in font rendering

self.font = pygame.font.SysFont('consolas', 20, False)

Problem: HTTP requests timing out

Solution:

```
// In agent.ts, add timeout handling
const controller = new AbortController();
const timeout = setTimeout(() => controller.abort(), 5000);

const res = await fetch(API_URL + '/data', {
signal: controller.signal
});
clearTimeout(timeout);
```

8.5 OmniLink Platform Issues

Problem: TypeScript files won't load in OmniLink

Solution:

- Verify file uses ESM syntax (no `require()`, use `import/export`)
- Check for Node.js-specific APIs (not available in browser)
- Use `console.log` for debugging instead of `process.stdout`

- Ensure MQTT.js is loaded via `globalThis.mqtt`

Problem: CORS errors in browser console

Solution:

- Verify server includes CORS headers (already configured in `server_wrapper.py`)
- Check network tab in DevTools for actual error
- If running on different machine, ensure firewall allows connections

---

9. Technical Details and Implementation Notes

9.1 Game Engine Architecture

Coordinate System:

- Board dimensions: 10 columns × 20 rows (standard Tetris)
- Cell size: 30 pixels
- Origin: Top-left (0,0)
- Piece spawn position: Center column (x=3), above visible area (y=-2)

State Machine:

State	Description
TITLE	Initial screen, waiting for start command
PLAY	Active gameplay, processing input and gravity
PAUSE	Frozen state, no updates
GAMEOVER	Final screen after all lives exhausted

Table 5: Game state machine

Collision Detection:

```
def valid(self, cells):
for (cx, cy) in cells:
if cx < 0 or cx >= COLS or cy >= ROWS:
```

```
return False # Out of bounds
if cy >= 0 and self.board[cy][cx]:
return False # Overlapping locked piece
return True
```

Lock Delay Mechanism:

Modern Tetris uses "infinity lock delay" where pieces can be moved on the ground indefinitely. This implementation uses a simplified 0.5-second lock delay that resets on horizontal moves or rotations.

9.2 Board State Representation

Board Array:

20 rows × 10 columns

Values: 0 = empty, 1-7 = color index for locked pieces

```
self.board = [[0]*COLS for _ in range(ROWS)]
```

Example board state with L-piece on left:

```
[[0,0,0,0,0,0,0,0,0,0], # Row 0 (top)
...
[0,0,0,0,0,0,0,0,0,0],
[4,0,0,0,0,0,0,0,0,3], # Row 18
[4,4,4,2,2,1,1,3,3,3]] # Row 19 (bottom)
```

Piece Representation:

```
piece = {
'type': 'T', # Piece type (I, O, T, S, Z, J, L)
'rot': 2, # Rotation index
'x': 4, # Column position
```

```
'y': 10 # Row position
}
```

JSON State Format (sent to agents):


```
{
"board": [[0,0,...], [0,0,...], ...],
"piece": {"type":"T", "rot":0, "x":4, "y":5, "numrotations":4},
"nextpiece": "I",
"score": 1200,
"hiscore": 15000,
"level": 3,
"lines": 25,
"lives": 14,
"playtime": 127.3,
"gamestate": "PLAY",
"cols": 10,
"rows": 20
}
```


9.3 Agent Decision Pipeline

Full Decision Cycle:

1. **State Acquisition:** Agent fetches JSON via GET /data
2. **Change Detection:** Checks if piece type changed from last cycle
3. **Placement Evaluation:** If new piece, evaluate all positions:
    - For each rotation (0 to numrotations-1)
    - For each valid column (bounded by piece shape)
    - Clone board array
    - Simulate hard-drop to ground
    - Lock piece and clear lines
    - Calculate heuristic score
4. **Best Selection:** Choose rotation and column with highest score
5. **Path Planning:** Generate macro-action sequence:
    - Rotate piece to target rotation
    - Move left/right to target column
    - Add DROP command
6. **Action Submission:** POST action array to /callback endpoint
7. **Delay:** Wait POLL_DELAY_MS before next cycle

9.4 Heuristic Calculation Details

Aggregate Height:

```
function aggregateHeight(board: number[][]): number {
let total = 0;
for (let c = 0; c < COLS; c++) {
for (let r = 0; r < ROWS; r++) {
if (board[r][c] !== 0) {
total += (ROWS - r); // Height from bottom
break;
}
}
}
return total;
}
```

Hole Counting:

```
function countHoles(board: number[][]): number {
let holes = 0;
for (let c = 0; c < COLS; c++) {
let filled = false;
for (let r = 0; r < ROWS; r++) {
if (board[r][c] !== 0) {
filled = true;
} else if (filled) {
holes++; // Empty cell below filled cell
}
}
}
return holes;
}
```

Bumpiness:

```
function bumpiness(heights: number[]): number {
let b = 0;
for (let i = 0; i < heights.length - 1; i++) {
b += Math.abs(heights[i] - heights[i+1]);
}
```

```
return b;
}
```

9.5 Performance Characteristics

Computational Complexity:

For each piece decision:

- Piece types: 7 (I, O, T, S, Z, J, L)
- Max rotations: 4 (T, S, Z, J, L), 2 (I), 1 (O)
- Max columns: ~10 per rotation
- Evaluations per piece: ~30-40 board simulations
- Time per evaluation: <1ms on modern hardware
- Total decision time: ~40-60ms

Network Performance:

- HTTP request latency: 1-5ms (localhost)
- JSON serialization: <1ms
- MQTT message latency: <1ms
- End-to-end agent cycle: 60-80ms

Game Performance:

- Rendering: 60 FPS (16.67ms per frame)
- Gravity updates: Variable (1000ms to 50ms depending on playtime)
- Action processing: 40ms per action (25 actions/second)
- Maximum sustained speed: ~20 pieces/second at endgame speeds

9.6 Scalability Considerations

Multi-Agent Support:

The architecture supports multiple agents simultaneously:

- Each agent polls independently
- Actions are queued atomically
- Version tracking prevents stale data processing
- MQTT telemetry broadcasts to all subscribers

Cloud Deployment:

For production deployment:

- Run game backend in Docker container
- Use managed MQTT broker (AWS IoT, Azure IoT Hub)
- Deploy agents as serverless functions
- Add authentication to HTTP endpoints
- Implement rate limiting to prevent abuse

9.7 Known Limitations

- **Single-piece lookahead:** Agent only considers current piece, not next piece (could be extended to 2-piece lookahead)
- **Static weights:** Heuristic weights are fixed (could adapt based on board state)
- **No T-spin detection:** Agent doesn't recognize or execute advanced Tetris techniques
- **Blocking architecture:** Game runs in single thread (could be parallelized)
- **No replay system:** Games cannot be recorded and replayed
- **Local high score only:** No global leaderboard integration

---

10. Conclusion and Future Enhancements

10.1 Project Summary

This project demonstrates a production-grade implementation of an AI-controlled gaming system with clear separation of concerns, robust communication protocols, and sophisticated decision algorithms. The architecture is extensible and suitable for educational purposes, research applications, or competitive AI development.

10.2 Potential Enhancements

Algorithmic Improvements:

- Implement 2-piece or 3-piece lookahead for better planning
- Add T-spin and combo detection for advanced techniques

- Use deep reinforcement learning (DQN, PPO) instead of heuristics
- Implement genetic algorithm for weight optimization

Feature Additions:

- Multiple game modes (marathon, sprint, ultra)
- Leaderboard and replay system
- Real-time visualization dashboard
- Multi-agent tournament mode
- Web-based control interface

Technical Improvements:

- Migrate to WebSocket for bi-directional communication
- Add authentication and authorization
- Implement distributed agent orchestration
- Add Prometheus metrics and Grafana dashboards
- Containerize with Docker Compose

10.3 Learning Outcomes

This project provides hands-on experience with:

- Service-oriented architecture design
- HTTP REST API implementation
- MQTT publish/subscribe messaging
- Heuristic algorithm development
- Real-time system synchronization
- Cross-language communication (Python ↔ TypeScript)
- Browser-based agent execution
- Game AI development
- Performance optimization techniques

Appendix A: Quick Reference Commands

Start Full System:

Terminal 1: MQTT Broker

mosquitto -c /etc/mosquitto/mosquitto.conf

Terminal 2: Game Backend

python server_wrapper.py

Terminal 3: AI Agent

npx ts-node advanced_agent.ts

Test Endpoints:

Get game state

curl http://localhost:5001/data

Send action

curl -X POST http://localhost:5001/callback
-H "Content-Type: application/json"
-d '{"actions":["LEFT","DROP"]}'

Monitor MQTT telemetry

mosquitto_sub -h localhost -t olinkcontext -v

Control Commands:

Pause game (MQTT)

mosquitto_pub -h localhost -t olinkcommands -m
'{"command":"pause"}'

Resume game (MQTT)

mosquitto_pub -h localhost -t olinkcommands -m '{"command":"resume"}'

Appendix B: Port Reference

Port	Protocol	Purpose
5001	HTTP	REST API (game state, action submission)
1883	MQTT	Message broker (standard TCP)
9001	MQTT/WS	Message broker (WebSocket for browsers)

Table 6: Network ports used by the system

Appendix C: Configuration Files

mosquitto.conf (minimum configuration):

```
listener 1883
protocol mqtt

listener 9001
protocol websockets

allow_anonymous true
```

tsconfig.json (if needed for TypeScript compilation):

```
{
"compilerOptions": {
"target": "ES2020",
"module": "ESNext",
"lib": ["ES2020", "DOM"],
"strict": true,
"esModuleInterop": true,
"skipLibCheck": true,
"forceConsistentCasingInFileNames": true
```

```
}
}
```

References

[1] README-3.md - OmniLink Tetris Base Project Architecture Documentation

[2] tetris-5.py - Pygame Tetris Implementation Source Code

[3] server_wrapper-4.py - HTTP REST API and MQTT Bridge Implementation

[4] advanced_agent.ts - TypeScript AI Agent with Pierre Dellacherie Algorithm

[5] agent-2.ts - Basic TypeScript AI Agent Implementation

[6] El-Tetris Algorithm Overview. (2024). Retrieved from https://imake.ninja/el-tetris-an-improvement-on-pierre-dellacheries-algorithm/

[7] MQTT Over WebSockets Configuration Guide. (2024). Steve's Internet Guide. https://www.steves-internet-guide.com/mqtt-websockets/

[8] Tetris AI Implementation using Pierre Dellacherie Algorithm. GitHub Repository. https://github.com/daogan/tetris-ai

---

Document Version: 1.0
Last Updated: March 10, 2026
Contact: For questions or issues, refer to project README.md or repository documentation